WSN Energy-Harvesting Routing Simulator

Full-Stack Web Application Execution & Setup Guide (FastAPI Backend + React Frontend)

GitHub Repository: https://github.com/santhoshvellore7119-web/WSN_proj | Branch: master | Author: Santhosh

1. Full-Stack System Architecture Overview

The system pairs an asynchronous FastAPI REST backend with a modern React 19 single-page application. The frontend provides an interactive SVG topology canvas, replay controls, and real-time residual energy charts.

Layer	Tech Stack & Port	Role & Responsibilities
Frontend UI	React 19, TypeScript, Vite, Tailwind CSS, Recharts **Port 3000**	Interactive dashboard: real-time SVG topology visualization, round replay controls, dynamic energy analytics.
Backend API	FastAPI, Uvicorn, SQLite, SQLAlchemy, Pydantic **Port 8000**	Async REST API: manages background simulation jobs, SQLite database persistence, and OpenAPI Swagger documentation.
Simulation Engine	Python 3.10+, NumPy, SciPy (Core Library in src/)	Executes 3D Time-Augmented DP ($dp[v][h][t]$), DSU live detour recovery, LEACH clustering, and solar irradiance traces.

CRITICAL WORKING DIRECTORY REQUIREMENT:
- **Project Root: C:\Users\Santhosh\wsn-energy-routing**
- **Frontend Folder: C:\Users\Santhosh\wsn-energy-routing\frontend**
- **CAUTION: Do NOT run commands inside C:\Users\Santhosh\frontend (which is an unrelated, separate project). Executing commands in the wrong directory will trigger *'Missing script: dev'* or *'requirements.txt not found'*.**

2. Prerequisites & One-Time Initial Setup

Ensure the following tools are installed: **Python 3.10+** (with pip), **Node.js 18+** (with npm), and **Git**.

Step 1: Open or Clone the Repository

```
cd C:\Users\Santhosh\wsn-energy-routing
# (If cloning for the first time: git clone https://github.com/santhoshvellore7119-web/WSN_proj.git wsn-energy-routing)
```

Step 2: Setup Python Virtual Environment (.venv) & Install Backend Dependencies

- Windows (CMD / PowerShell):

```
cd C:\Users\Santhosh\wsn-energy-routing
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
```

(Note: Always use python -m pip install to prevent Windows binary-locking update errors)

Step 3: Install Frontend Node Dependencies

```
cd C:\Users\Santhosh\wsn-energy-routing\frontend
npm install
cd ..
```

3. How to Run the Full-Stack Application

Method A: Automated 1-Click Startup (Recommended)

A pre-configured startup script is included in the project root to automatically manage ports, activate environments, and launch both servers:

```
cd C:\Users\Santhosh\wsn-energy-routing
.\start_offline.bat
```

(Or simply double-click start_offline.bat in Windows File Explorer. It cleans ports 8000/3000, starts FastAPI, starts React, and opens http://localhost:3000 in your browser).

Method B: Manual Startup in Two Terminals

- Terminal 1 – Backend (Port 8000):

```
cd C:\Users\Santhosh\wsn-energy-routing
.venv\Scripts\activate
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000
```

- Terminal 2 – Frontend (Port 3000):

```
cd C:\Users\Santhosh\wsn-energy-routing\frontend
npm run dev
```

- *Dashboard:* **http://localhost:3000** | *Swagger API Docs:* **http://127.0.0.1:8000/docs**

4. Step-by-Step Procedure to Run in Visual Studio Code (VS Code)

Follow these steps for a clean, integrated development workflow inside VS Code:

Step 1: Open Project in VS Code

Open terminal and launch VS Code at the project root: `cd C:\Users\Santhosh\wsn-energy-routing && code .`

Step 2: Select Python Interpreter

1. Press Ctrl + Shift + P (or Cmd + Shift + P on macOS).
2. Type and select **Python: Select Interpreter**.
3. Choose the workspace virtual environment: .\.venv\Scripts\python.exe.

Step 3: Open Integrated Split Terminals

1. Open the terminal panel with Ctrl + ` (backtick).
2. Click the **Split Terminal** icon on the top right (or press Ctrl + \).
3. In the **Left Pane (Backend)**, run:
   ```
   .\.venv\Scripts\activate && python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000
   ```
4. In the **Right Pane (Frontend)**, run:
   ```
   cd frontend && npm run dev
   ```

Step 4: Optional 1-Click Debugging via .vscode/launch.json

Configure the VS Code debugger for breakpoint inspection by adding to .vscode/launch.json:

```
{ "version": "0.2.0", "configurations": [ { "name": "Debug FastAPI Backend", "type": "debugpy", "request": "launch",
"module": "uvicorn", "args": ["backend.main:app", "--reload", "--port", "8000"], "jinja": true } ] }
```

5. Running with Docker Compose (1-Command Full-Stack Boot)

If you have Docker Desktop installed, boot both containers simultaneously without configuring local environments:

```
# Build and start all containers in detached mode:
docker compose up --build -d

# View live logs from both services:
docker compose logs -f

# Stop and clean up containers:
docker compose down
```

• **Web Dashboard:** http://localhost:3000 | • **FastAPI Swagger Docs:** http://127.0.0.1:8000/docs

6. Key REST API Endpoints Overview

Method & Route	Request Body / Params	Functionality
`POST /api/run-simulation`	JSON: { num_nodes, rounds, harvesting_profile, enable_time_dp, ... }	Triggers asynchronous simulation in background task. Returns job ID.
`GET /api/simulation/{id}`	Path param: job_id (UUID)	Polls live execution status, node topology coordinates, and round telemetry.
`GET /api/scenarios`	None	Returns pre-configured scenario templates (Baseline, Solar, Shadowed, Stochastic).
`GET /api/history`	Query: limit, offset	Retrieves paginated past simulation runs stored in SQLite.
`GET /api/export/{id}/csv`	Path param: job_id	Exports round-by-round energy, alive nodes, and packet metrics as CSV.

7. Troubleshooting & Common Issues

Symptom / Error	Underlying Cause	Resolution
npm error Missing script: "dev"	Command was run in C:\Users\Santhosh\frontend (unrelated folder) rather than the project frontend.	Navigate to cd C:\Users\Santhosh\wsn-energy-routing\frontend and verify package.json contains "dev": "tsx server.ts".
ERROR: Could not open requirements file: 'requirements.txt'	Terminal is inside frontend\ or outside project. requirements.txt is in the project root.	Run cd C:\Users\Santhosh\wsn-energy-routing before executing pip install -r requirements.txt.
ERROR: To modify pip, please run python.exe -m pip...	Windows file-locking prevents running pip.exe from modifying its own executable in-place.	Always use the module syntax: python -m pip install --upgrade pip
PowerShell execution policy blocks activate.ps1	Default Windows PowerShell restriction blocks local script execution.	Execute once in PowerShell: Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
Port 8000 or 3000 already in use	Previous uvicorn or node server processes are still occupying the ports.	Run start_offline.bat (auto-cleans ports) or manually: netstat -ano | findstr :8000 then taskkill /PID <PID> /F
502 Bad Gateway / Network Error in Browser	Frontend proxy cannot connect to backend because FastAPI is offline.	Ensure the FastAPI backend is running on port 8000 before opening the frontend in your browser.

8. Quick Full-Stack Execution Cheat-Sheet

```
# ------------------------------------------------------------
# OPTION A: ONE-CLICK LAUNCH (FASTEST)
# ------------------------------------------------------------
cd C:\Users\Santhosh\wsn-energy-routing
.\start_offline.bat

# ------------------------------------------------------------
# OPTION B: MANUAL TWO-TERMINAL SETUP
# ------------------------------------------------------------
# Terminal 1 (FastAPI Backend - Port 8000):
cd C:\Users\Santhosh\wsn-energy-routing
.venv\Scripts\activate
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000

# Terminal 2 (React Frontend - Port 3000):
cd C:\Users\Santhosh\wsn-energy-routing\frontend
npm run dev

# Access in Browser:
# Dashboard: http://localhost:3000 | API Docs: http://127.0.0.1:8000/docs
```